



SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 1 of 15 •



50 Most Important System Design Questions with Simple Answers, Diagrams & Tips!



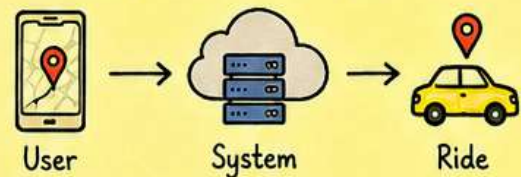
1 WHAT IS SYSTEM DESIGN?

System Design is the process of designing large-scale, reliable, scalable and efficient systems to meet given requirements.

Goal → Build systems that are Scalable, Reliable, Available & Maintainable.

EXAMPLE

Design a Ride Booking System



Take requirements → Design → Break down → Build → Scale

2 FUNCTIONAL vs NON-FUNCTIONAL REQUIREMENTS

FUNCTIONAL (What the system does)

- User Registration & Login
- Search for Driver
- Book / Cancel Ride
- Real-time Tracking
- Payment
- Ride History

NON-FUNCTIONAL

(How well the system does it)

- High Availability (99.9% uptime)
- Low Latency (< 200ms)
- Scalability (Handle millions of users)
- Security
- Reliability & Fault Tolerance

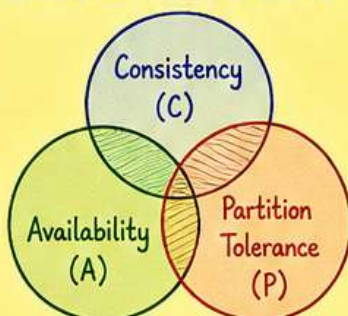
INTERVIEW TIP



- ★ Always clarify requirements before jumping into design.

3 CAP THEOREM

In a distributed system, you can achieve at most two out of three.



- ★ **Consistency:** Every read receives the most recent write or an error.

- ★ **Availability:** Every request receives a response.

- ★ **Partition Tolerance:** The system continues to operate despite network partitions.

PRACTICAL TAKE

- CP System = Consistency + Partition Tolerance (e.g., HBase, MongoDB)
- AP System = Availability + Partition Tolerance (e.g., Cassandra, DynamoDB)
- CA System = Consistency + Availability (Only in single node)



REMEMBER: There is no one-size-fits-all. Choose based on your use case! ♥



SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 2 of 15 •



Understand these fundamentals deeply. They are the foundation of any scalable system!



4 CONSISTENCY vs AVAILABILITY

CONSISTENCY (C)

All nodes see the same data at the same time.

Example:

Bank balance after money transfer.

AVAILABILITY (A)

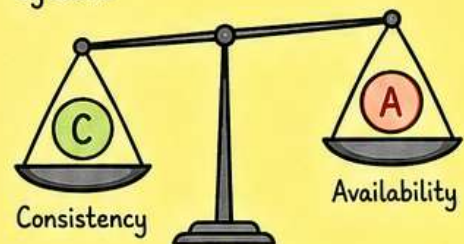
Every request receives a response, even if it is the latest data.

Example:

Always showing some response to user.

TRADE-OFF

You usually have to choose one over the other in a distributed system.



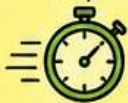
Most real-world systems aim for "Eventual Consistency" (A + some delay in C)



5 LATENCY vs THROUGHPUT

LATENCY

Time taken for a request to go from client to server and get a response.



Goal: Reduce Latency

THROUGHPUT

Number of requests a system can handle in a given time.



Goal: Increase Throughput

KEY POINTS

- ★ Low latency → Better user experience
- ★ High throughput → Handle more users
- ★ Improving throughput can increase latency
- ★ Both must be balanced based on use case



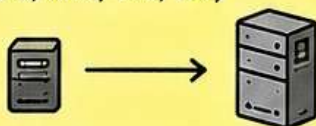
Measure both and optimize based on business priorities!



6 SCALABILITY (VERTICAL vs HORIZONTAL)

VERTICAL SCALING (Scale Up)

Increase the power of a single server (CPU, RAM, Disk, etc.)



- ✓ Pros: Simple to implement
- ✗ Cons: Expensive, has hardware limits, single point of failure

HORIZONTAL SCALING (Scale Out)

Add more servers to handle increased load.



- ✓ Pros: Cost-effective, fault tolerant, handles massive scale
- ✗ Cons: More complex to design & maintain

SCALING STRATEGIES

- Load Balancing
- Database Sharding
- Caching
- CDN
- Asynchronous Processing
- Auto Scaling



Design for scale from day one!



REMEMBER: Good design = Right trade-offs + Right patterns + Right measurements!



2



SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 3 of 15 •

★ Databases are the backbone of most applications.
Know how to store, scale & keep data safe! ❤️

7 SQL vs NoSQL

SQL (Relational)

- Table based (Rows & Columns)
- Fixed Schema
- ACID compliant
- Joins supported
- Best for complex queries
- Examples: MySQL, Postgres

NoSQL (Non-Relational)

- Key-Value, Document, Column, Graph, etc.
- Flexible Schema
- BASE (Eventually Consistent)
- No joins
- Best for scalability
- Examples: MongoDB, Cassandra, Redis, DynamoDB

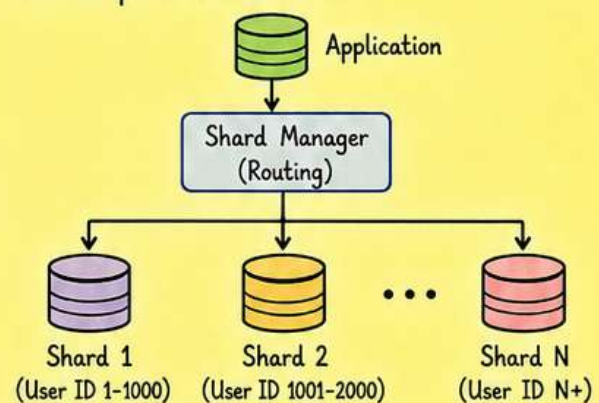


Use SQL for structured data & complex queries.

Use NoSQL for scale & flexibility. ❤️

8 DATABASE SHARDING

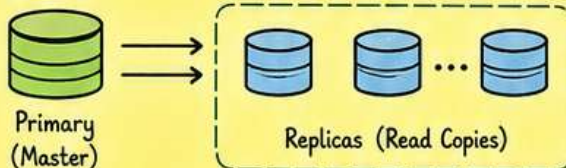
Sharding splits a large database into smaller parts called shards.



★ Benefits: Horizontal scaling, Faster queries, Better performance. ❤️

9 REPLICATION

Replication means keeping multiple copies of data for high availability and fault tolerance.



TYPES

- Master-Slave Replication (Primary-Replica)
- Multi-Master Replication

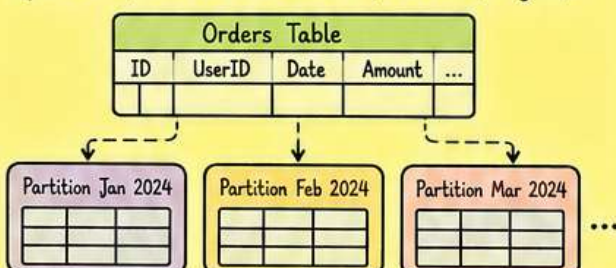
BENEFITS

- ✓ High Availability
- ✓ Read Scalability
- ✓ Data Backup & Disaster Recovery



10 PARTITIONING

Partitioning divides a table into smaller pieces (partitions) based on a column (like date, region).



ADVANTAGES

- ✓ Improves query performance
- ✓ Easy data management
- ✓ Old data can be archived or deleted
- ✓ Reduces load on single partition



Choose partition key wisely (e.g., Date, UserID, Region). ❤️

EXAMPLE

E-commerce orders table partitioned by month.



REMEMBER: Right Database + Right Design = Scalable & Reliable System! ❤️

SYSTEM DESIGN

INTERVIEW QUESTIONS



• Page 4 of 15 •

★ Caching makes your system faster, reduces DB load and improves overall performance! ❤️

11 WHY REDIS?

- ✓ In-memory data structure store
- ✓ Extremely fast (μ s level)
- ✓ Supports rich data structures (String, Hash, List, Set, ZSet, etc.)
- ✓ Built-in persistence
- ✓ High availability (Replication, Sentinel, Cluster)
- ✓ Great for caching, sessions, rate limiting, leaderboards

COMMON USE CASES

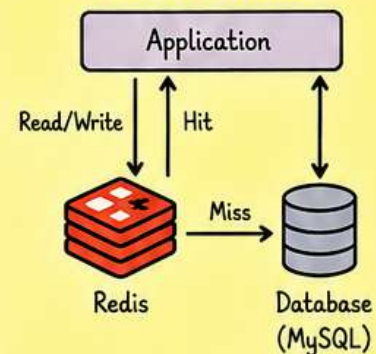
- Caching
- Session Store
- Leaderboards
- Rate Limiting
- Counters
- Real-time Analytics



INTERVIEW TIP

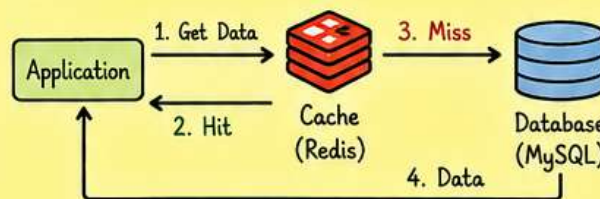
Redis is not just a cache, it's an in-memory database! ❤️

HOW IT WORKS



12 CACHE ASIDE PATTERN

- 1 Application checks cache.
- 2 If hit → return data.
- 3 If miss → fetch from DB.
- 4 Store in cache & return.



PROS

- ✓ Simple to implement
- ✓ High performance
- ✓ Cache has full control

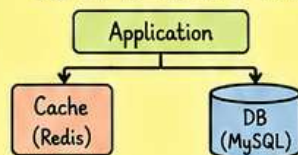
CONS

- ✗ More code
- ✗ Cache misses add extra latency

13 WRITE THROUGH vs WRITE BACK

WRITE THROUGH

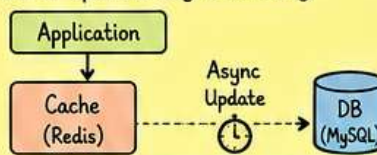
Data is written to cache and DB at the same time.



- ✓ Strong consistency
- ✗ Slower (two writes)

WRITE BACK (WRITE BEHIND)

Data is written to cache first. DB is updated asynchronously.



- ✓ Faster writes
- ✗ Data loss risk if cache crashes

INTERVIEW TIP



Use Write Through for critical data.
Use Write Back for performance critical systems. ❤️

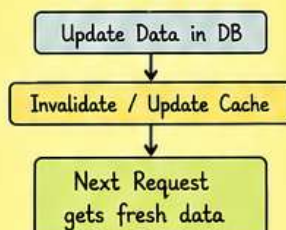
14 CACHE INVALIDATION

Keeping cache and DB in sync is very important.

COMMON STRATEGIES

- TTL (Time To Live)
- Manual Invalidation
- Write Through / Write Back
- Versioning
- Event Based Invalidation

EXAMPLE FLOW



WHY IMPORTANT?

Stale data in cache can lead to wrong results and bad user experience. ⚠️

★ BEST PRACTICE

Always think about "When and How" to invalidate cache. There is no one-size-fits-all strategy! ❤️



REMEMBER: Cache is a tool. Use it wisely to get the best performance! ❤️



SYSTEM DESIGN

INTERVIEW QUESTIONS

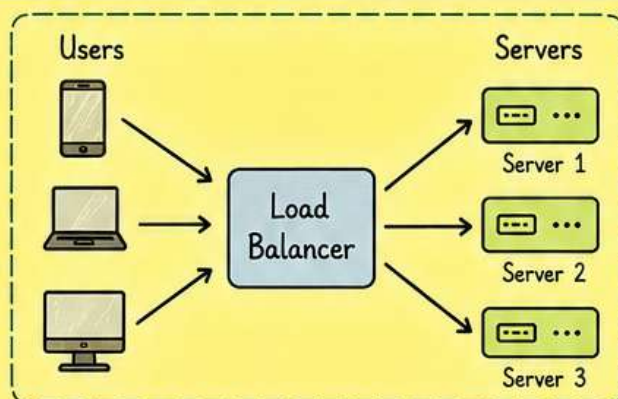
• Page 5 of 15 •

★ Load Balancing distributes incoming traffic across multiple servers to improve performance and reliability! ❤️

15 WHAT IS LOAD BALANCER?

A Load Balancer sits between client requests and servers. It distributes the requests across multiple servers to ensure:

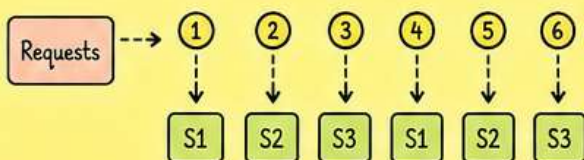
- ✓ High Availability
- ✓ Scalability
- ✓ Better Performance
- ✓ Fault Tolerance



One entry point → Many servers → Happy users! ❤️

16 ROUND ROBIN

Requests are distributed to servers one by one in a cyclic order.



PROS

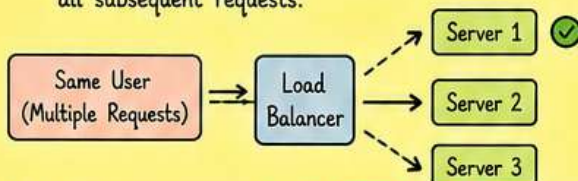
- ✓ Simple to implement
- ✓ Fair distribution

CONS

- ✗ Doesn't consider server capacity
- ✗ Can overload slower servers

18 STICKY SESSIONS (SESSION AFFINITY)

A user is always routed to the same server for all subsequent requests.



PROS

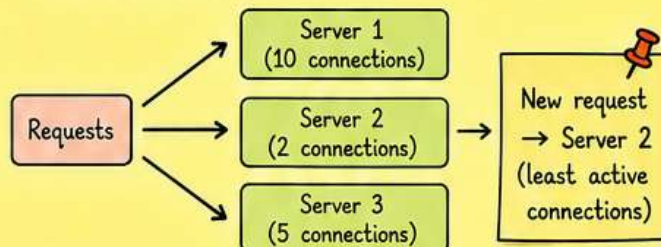
- ✓ Better user experience
- ✓ Useful for session-based applications

CONS

- ✗ Uneven load distribution
- ✗ Not ideal for stateless applications

17 LEAST CONNECTIONS

Requests are sent to the server with the least active connections.



PROS

- ✓ Better performance
- ✓ Considers server load
- ✓ Avoids overloading

CONS

- ✗ Slightly complex
- ✗ Needs connection tracking

INTERVIEW TIP

★ Always clarify the use case before choosing the algorithm.

★ For most real-world systems, we use Round Robin + Health Checks! 😊



REMEMBER: A good Load Balancer = Happy Users + Healthy Servers! ❤️



SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 6 of 15 •



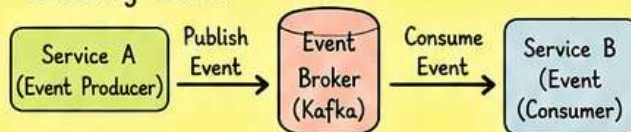
Communication between services in distributed systems should be decoupled, reliable and scalable! ❤️

19 KAFKA vs RABBITMQ

KAFKA	RABBITMQ
• Distributed Streaming Platform	• Message Broker
• High Throughput	• Lower Throughput
• Pull Based	• Push Based
• Partitioned Topics	• Queues
• Best for Event Streaming, Logs, Analytics	• Best for Task Queues, Background Jobs
• Durable by Design	• Depends on Configuration
• Examples: LinkedIn, Uber, Netflix	• Examples: Stripe, Shopify, User Management

20 EVENT-DRIVEN ARCHITECTURE

Components communicate by producing and consuming events.



BENEFITS

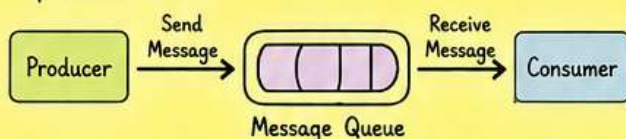
- ✓ Loose Coupling
- ✓ Scalability
- ✓ Resilience
- ✓ Real-time Processing
- ✓ Easy to extend new services

USE CASES

- ✓ Order placed → Send Email
- ✓ Payment success → Update inventory
- ✓ User signed up → Welcome email

21 MESSAGE QUEUE

A queue that holds messages until a consumer processes them.



ADVANTAGES

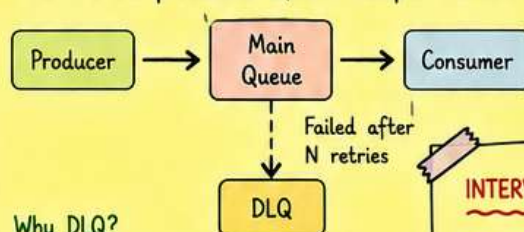
- ✓ Decouples Producer & Consumer
- ✓ Improves Reliability
- ✓ Handles Traffic Spikes
- ✓ Asynchronous Communication

EXAMPLE TOOLS

- Kafka
- RabbitMQ
- SQS (AWS)
- Google Pub/Sub
- ActiveMQ

22 DEAD LETTER QUEUE (DLQ)

A special queue that stores messages that cannot be processed after multiple retries.



Why DLQ?

- ✓ Prevent message loss
- ✓ Analyze failed messages
- ✓ Reprocess after fixing issues

INTERVIEW TIP

Always handle failures gracefully & monitor DLQ! 😊

23 REST vs GRAPHQL

REST	GRAPHQL
Multiple Endpoints	Single Endpoint
Over-fetching	Fetch only what you need
Versioning is complex	No versioning
Strongly typed contract	Flexible queries
Best for simple CRUD apps	Best for complex & mobile apps

GRAPHQL BENEFITS

- Reduce over-fetching & under-fetching
- Better performance
- Strongly typed
- Client gets exact data required

24 IDEMPOTENCY

The property of an operation to produce the same result even if it is called multiple times.

EXAMPLE: Payment API



How to achieve idempotency?

- ✓ Idempotency Key (Unique for each request)
- ✓ Store response & return same result for duplicate calls



REMEMBER: Decouple, Communicate, Handle failures & Monitor everything! ❤️

SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 7 of 15 •



APIs are the interface of your system. Design them clean, secure, consistent and scalable!



23 REST vs GRAPHQL

	REST	GRAPHQL
Multiple Endpoints	Yes (Many URLs)	No (Single Endpoint)
Data Fetching	Over-fetching Under-fetching	Fetch exactly what you need
Versioning	Required (v1, v2...)	Not required
Strongly Typed	No	Yes (Schema based)
Caching	Easy (HTTP cache)	Depends on implementation
Best For	Simple CRUD, Public APIs	Complex, Mobile, Multi-client apps



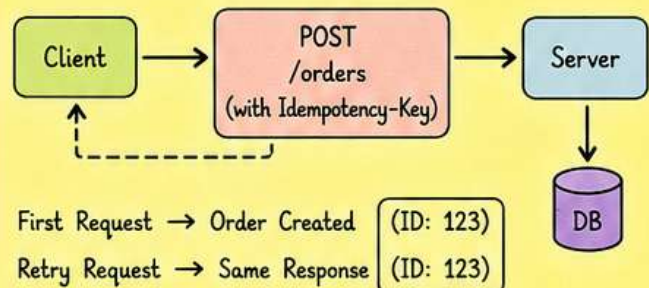
INTERVIEW TIP

Use REST for simple resources.
Use GraphQL when clients need flexibility and fewer network calls. ❤️

24 IDEMPOTENCY

An idempotent operation returns the same result even if it is called multiple times.

EXAMPLE: Create Order API



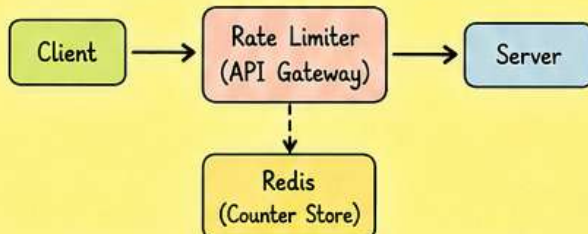
HOW TO IMPLEMENT?

- ✔ Use Idempotency Key (Unique per request)
- ✔ Store key + response in DB/Cache
- ✔ Return stored response for duplicate requests

25 RATE LIMITING

Rate limiting restricts the number of requests a user can make in a given time window.

EXAMPLE: 100 requests per minute per user



STRATEGIES

- Fixed Window
- Sliding Window
- Token Bucket
- Leaky Bucket



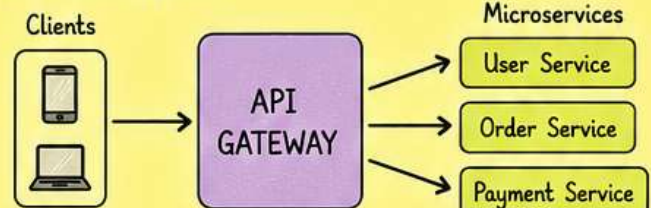
BEST PRACTICES

- ✔ Limit based on user/IP/API key
- ✔ Return 429 Too Many Requests
- ✔ Use Retry-After header
- ✔ Allow burst with Token Bucket



26 API GATEWAY

API Gateway acts as a single entry point for all client requests.



RESPONSIBILITIES

- ✔ Request Routing
- ✔ Authentication & Authorization
- ✔ Rate Limiting
- ✔ Caching
- ✔ Logging & Monitoring
- ✔ Request/Response Transformation
- ✔ Security (SSL, WAF)

POPULAR GATEWAYS

- Kong
- Nginx
- Zuul
- AWS API Gateway
- Zuul / Spring Cloud Gateway



REMEMBER: Design APIs simple, secure & scalable. Your API is your product! ❤️

SYSTEM DESIGN

INTERVIEW QUESTIONS



• Page 8 of 15 •

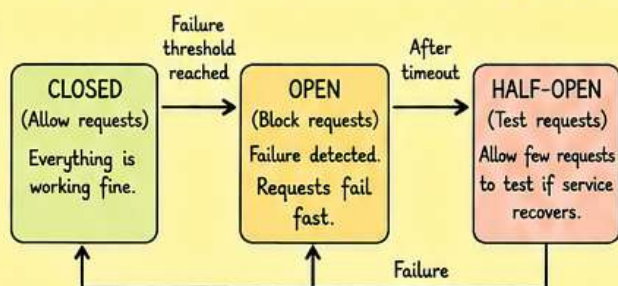


Reliability patterns help your system handle failures gracefully and keep it available!



27 CIRCUIT BREAKER

A circuit breaker prevents a system from repeatedly trying to execute an operation that is likely to fail.



PROS

- ✓ Prevent cascading failures
- ✓ Reduce load on failing service
- ✓ Improve system stability

CONS

- ✗ Complex to implement
- ✗ Needs careful tuning
- ✗ May block legit traffic

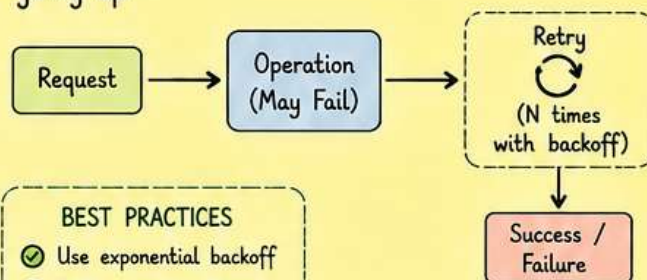


INTERVIEW TIP: Use Circuit Breaker with fallback to give a graceful user experience.



28 RETRY PATTERN

Retry the failed operation a few times before giving up.



BEST PRACTICES

- ✓ Use exponential backoff
- ✓ Set max retry attempts
- ✓ Add jitter to avoid thundering herd
- ✓ Retry only for idempotent operations

WHEN NOT TO RETRY?

- ✗ Non-idempotent operations
- ✗ Client errors (4xx)
- ✗ Business validation failures

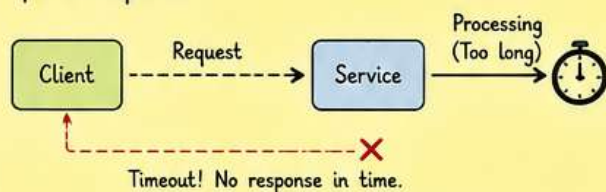


INTERVIEW TIP: Combine Retry with Timeout and Circuit Breaker.



29 TIMEOUT

A timeout ensures a system doesn't wait forever for a response.



WHY IMPORTANT?

- ✓ Avoids resource blocking
- ✓ Better user experience
- ✓ Improves system throughput

TYPES

- Connection Timeout
- Read Timeout
- Write Timeout

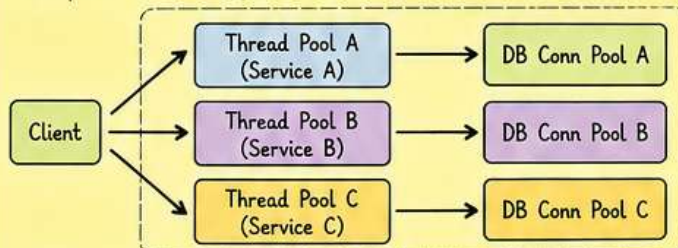


INTERVIEW TIP: Always set appropriate timeouts for external calls.



30 BULKHEAD PATTERN

Isolate failures by dividing resources so that failure in one part doesn't affect others.



BENEFITS

- ✓ Fault isolation
- ✓ Protects critical resources
- ✓ Improves resilience
- ✓ Better resource utilization

EXAMPLE

Different thread pools or DB connections for different services.



INTERVIEW TIP: Use Bulkhead with Circuit Breaker for strong isolation.



REMEMBER: Resilient systems don't avoid failures, they prepare for them!





SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 9 of 15 •



Microservices enable scalability and agility, but bring complexity. Design them wisely!



31 MONOLITH vs MICROSERVICES

MONOLITH	MICROSERVICES
✓ Single deployable unit	✓ Multiple small deployable services
✓ Easy to develop & test	✓ Independent development & deployment
✗ Hard to scale (scale whole app)	✓ Easy to scale (scale individual services)
✗ Tight coupling	✓ Loose coupling
✗ Technology stack is same	✓ Different tech stack per service
✓ Simple architecture	✗ Complex (network, data, ops)

WHEN TO USE?

Monolith:

- Small team
- Simple domain
- Early stage product

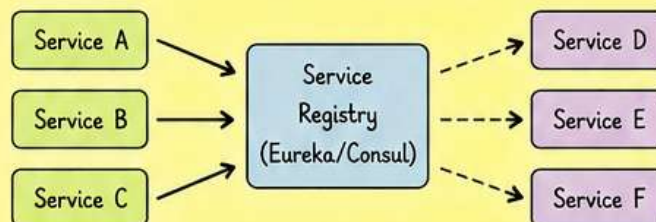
Microservices:

- Large product
- Multiple teams
- Need to scale fast



32 SERVICE DISCOVERY

Service Discovery helps services find and communicate with each other dynamically.



HOW IT WORKS

- ① Services register themselves with registry.
- ② Services discover other services from registry.
- ③ Communication happens using discovered address.

BENEFITS

- ✓ Decouples services
- ✓ Handles dynamic scaling
- ✓ No hardcoded endpoints
- ✓ Improves resilience

EXAMPLE TOOLS

- Netflix Eureka
- Consul
- Zookeeper
- Kubernetes DNS



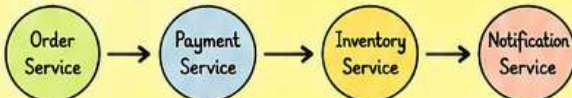
INTERVIEW TIP:

Explain heartbeat & TTL for healthy services.



33 SAGA PATTERN

Saga is used to manage distributed transactions across multiple services.



FLOW

- ① Each service performs local transaction.
- ② If all succeed → transaction completed.
- ③ If any fail → compensating transactions (rollback steps).

TYPES

- ① Choreography
Events drive the flow.
- ② Orchestration
Central entity orchestrates.

PROS

- ✓ Keeps system consistent
- ✓ Good for long running transactions
- ✓ Improves scalability

CONS

- ✗ Complex to design
- ✗ Hard to test
- ✗ Eventual consistency (not immediate)

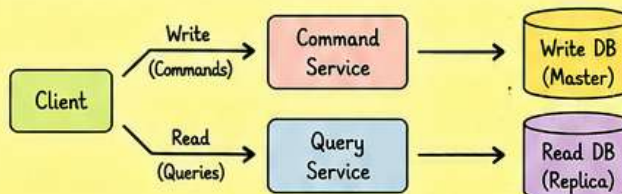


INTERVIEW TIP: Compare Saga with 2PC.



34 CQRS (COMMAND QUERY RESPONSIBILITY SEGREGATION)

CQRS separates read (query) and write (command) operations for better performance and scalability.



BENEFITS

- ✓ Optimized read & write models
- ✓ Scalability (scale reads & writes independently)
- ✓ Better performance

TRADE-OFFS

- ✗ Complex architecture
- ✗ Eventual consistency
- ✗ More operations overhead



INTERVIEW TIP: CQRS is often combined with Event Sourcing.



REMEMBER: Choose the right architecture & patterns based on your requirements, scale and team! ❤️



SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 10 of 15 •

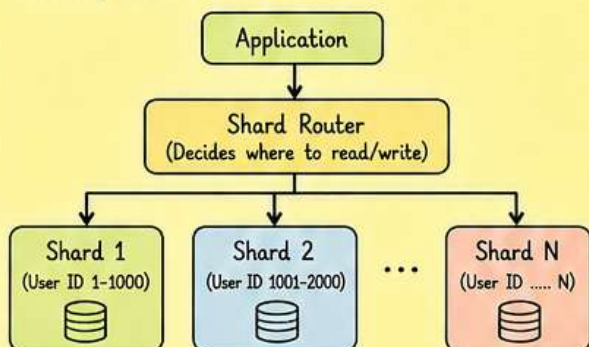


Good design is about managing complexity while delivering value to users!



35 SHARDING

Sharding is a database horizontal partitioning technique to scale databases.



PROS

- ✓ Better read/write throughput
- ✓ Handles large data
- ✓ Improved availability

CONS

- ✗ Complexity in design
- ✗ Cross-shard queries hard
- ✗ Rebalancing is tricky

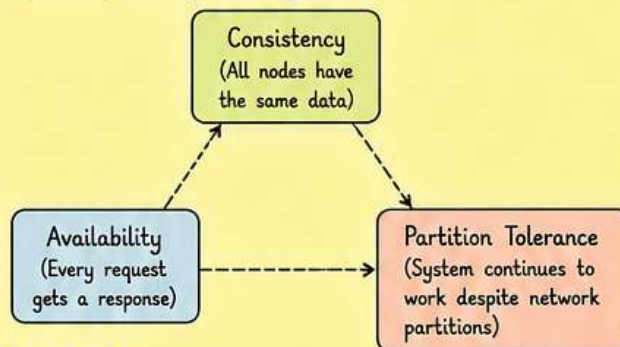


INTERVIEW TIP: Choose shard key carefully.
Good key = even distribution + query efficiency.



36 CAP THEOREM

In a distributed system, you can have at most two of the following three guarantees.



EXAMPLES

- CA System: RDBMS (not partition tolerant)
- CP System: HBase, MongoDB (consistency + partition tolerance)
- AP System: Cassandra, DynamoDB (availability + partition tolerance)

KEY TAKEAWAY

Pick the right trade-off based on your system and requirements.



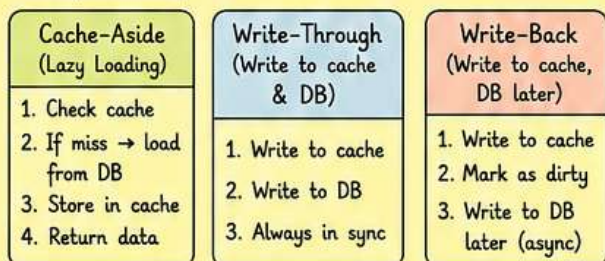
INTERVIEW TIP

Explain with real system examples to show clear understanding.



37 CACHING STRATEGIES

Caching stores frequently accessed data in memory to improve response time and reduce load.



BENEFITS

- ✓ Faster responses
- ✓ Reduces DB load
- ✓ Improves scalability

CHALLENGES

- ✗ Cache invalidation
- ✗ Stale data
- ✗ More memory usage

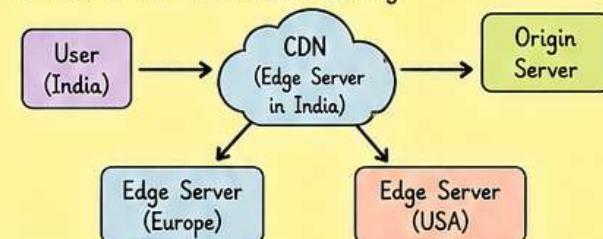


INTERVIEW TIP: Talk about trade-offs & use real-world cache.



38 CONTENT DELIVERY NETWORK (CDN)

CDN delivers content to users from servers nearest to them to reduce latency.



BENEFITS

- ✓ Reduced latency
- ✓ Handles high traffic
- ✓ Offloads origin server
- ✓ Better availability

EXAMPLES

- Cloudflare
- Akamai
- AWS CloudFront
- Fastly



INTERVIEW TIP: Mention static + dynamic content, caching & invalidation.



REMEMBER: The best system design is simple, scalable and solves real user problems! ♥



SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 11 of 15 •



Data is the new oil, but architecture is the engine that makes it valuable!

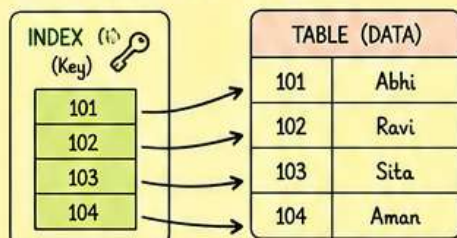


39 DATABASE INDEXING

Indexing improves the speed of data retrieval operations on a database table.

HOW IT WORKS

Index → Pointer to actual data



BENEFITS

- ✓ Faster read operations
- ✓ Efficient search & sort
- ✓ Unique constraints
- ✓ Improves query performance

TRADE-OFFS

- ✗ Slower write operations
- ✗ Consumes extra storage
- ✗ Index maintenance overhead



INTERVIEW TIP: Understand types of indexes (Primary, Secondary, Composite, Clustered).

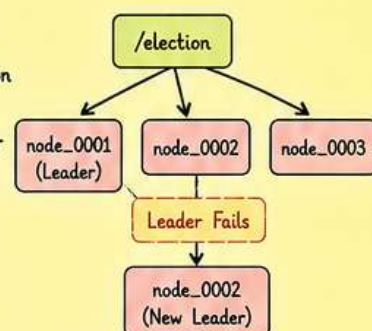


40 LEADER ELECTION (ZOOKEEPER)

Leader election is the process of selecting a single leader among distributed nodes to coordinate tasks.

HOW ZOOKEEPER DOES IT

- 1 All nodes create ephemeral sequential nodes under /election
- 2 The node with the smallest sequence number becomes leader
- 3 Others watch the leader node
- 4 If leader fails, next smallest node becomes the leader



BENEFITS

- ✓ Highly available
- ✓ Fault tolerant
- ✓ Strong consistency
- ✓ Simple & reliable

USE CASES

- Distributed locks
- Configuration management
- Leader based processing
- Failover handling



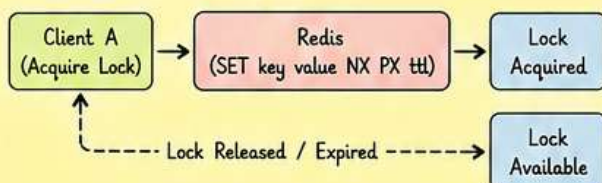
INTERVIEW TIP: Know concepts like ephemeral nodes, watches & ZAB protocol.



41 DISTRIBUTED LOCK

A distributed lock ensures that only one process or node can access a shared resource at a time.

HOW IT WORKS (Using Redis)



BEST PRACTICES

- ✓ Set expiration (TTL) always
- ✓ Use unique value (UUID)
- ✓ Release lock safely (Lua script)
- ✓ Handle lock renewal if needed

USE CASES

- Cron jobs
- Inventory updates
- Distributed schedulers
- Payment processing



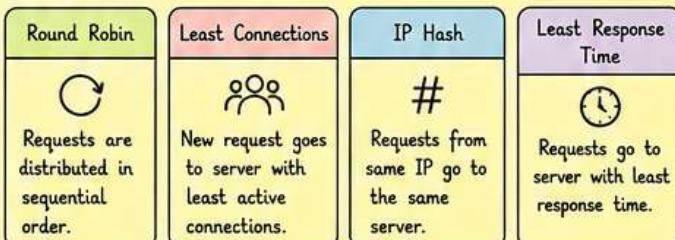
INTERVIEW TIP: Discuss Redlock algorithm and its safety guarantees.



42 LOAD BALANCING

Load balancing distributes incoming traffic across multiple servers to ensure no single server is overloaded.

TYPES



BENEFITS

- ✓ High availability
- ✓ Scalability
- ✓ Better performance
- ✓ Fault tolerance

COMPONENTS

- Load Balancer (e.g., Nginx, HAProxy)
- Server Pool
- Health Check
- Session Management (if needed)



INTERVIEW TIP: Understand L4 (TCP) vs L7 (HTTP) load balancing.



REMEMBER: Great systems balance performance, reliability & simplicity!

11

SYSTEM DESIGN

INTERVIEW QUESTIONS

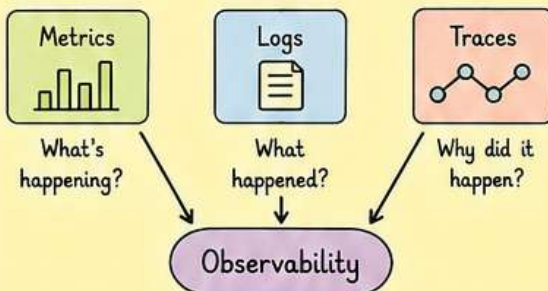


• Page 12 of 15 •

★ Observability turns outages into insights and guesswork into actions! ❤️

43 MONITORING & OBSERVABILITY

Monitoring tells you "something is wrong".
Observability tells you "why it's wrong".



KEY METRICS

- ✓ Latency (p50, p95, p99)
- ✓ Throughput (RPS)
- ✓ Error Rate
- ✓ Saturation (CPU, Mem, Disk)

TOOLS

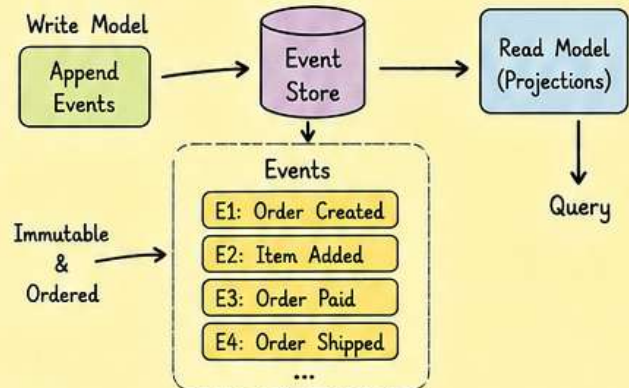
- Prometheus + Grafana
- ELK / Loki
- Jaeger / Zipkin
- Datadog / New Relic



INTERVIEW TIP: Explain the 3 pillars - Metrics, Logs and Traces with a real example. 🛡️

44 EVENT SOURCING

Store state changes as a sequence of events.
Current state is derived from events.



BENEFITS

- ✓ Audit trail
- ✓ Rebuild state
- ✓ Time travel / history
- ✓ Decoupled systems

DRAWBACKS

- Complex to implement
- Harder querying
- Event versioning



INTERVIEW TIP: When to use Event Sourcing over CRUD? 🛡️

45 MESSAGE QUEUES

Decouple services, handle spikes and enable async communication.



BENEFITS

- ✓ Decoupling
- ✓ Async processing
- ✓ Buffering / smoothing spikes
- ✓ Reliability (retries)

USE CASES

- Email notifications
- Order processing
- Logs aggregation
- Background jobs

POPULAR QUEUES

- Kafka (High throughput, durable)
- RabbitMQ (Flexible, widely used)
- SQS (AWS managed)
- Azure Service Bus

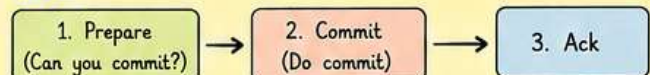


INTERVIEW TIP: Explain at-least-once vs at-most-once delivery. 🛡️

46 DISTRIBUTED TRANSACTION (2PC)

Two Phase Commit ensures all-or-nothing across multiple services/databases.

FLOW



COMMIT PATH

All say YES
↓
All commit

ABORT PATH

Any says NO
↓
All rollback

PROS

- ✓ Ensures consistency
- ✓ Simple to understand
- ✓ Supported in DBs

CONS

- Blocking (holds locks)
- Single point of failure
- Poor availability



INTERVIEW TIP: Compare 2PC with Saga pattern. 🛡️



REMEMBER: Design for scale, build for failure, monitor everything! ❤️



SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 13 of 15 •



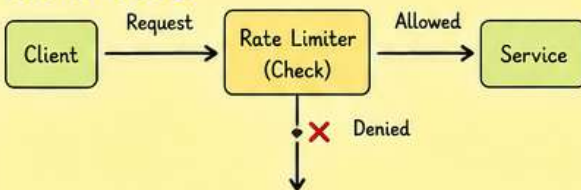
Great systems are built with good design, clear thinking, and the right trade-offs!



47 RATE LIMITER

Controls the number of requests a user can make in a given time window.

HOW IT WORKS



BENEFITS

- ✓ Prevents abuse
- ✓ Protects services
- ✓ Fair usage for all

ALGORITHMS

- Fixed Window Counter
- Sliding Window Log
- Token Bucket
- Leaky Bucket

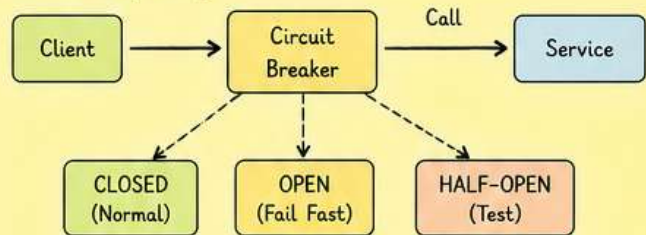


INTERVIEW TIP: Implement with Redis or in-memory cache for high performance.



48 CIRCUIT BREAKER

Prevents cascading failures by stopping calls to a failing service temporarily.



BENEFITS

- ✓ Improves system stability
- ✓ Prevents overload
- ✓ Allows recovery

STATES

- CLOSED – All requests allowed
- OPEN – Requests blocked
- HALF-OPEN – Test requests



INTERVIEW TIP: Combine with retry logic and timeout for best results.



49 LEAST RECENTLY USED (LRU) CACHE

Stores recently accessed items and evicts the least recently used item when full.

HOW IT WORKS



BENEFITS

- ✓ Fast access
- ✓ Widely used
- ✓ Small memory footprint

USE CASES

- Database Query Cache
- Web Cache
- DNS Cache



INTERVIEW TIP: Use a hash map + doubly linked list for O(1) get and put operations.



50 SYSTEM DESIGN PROCESS

A structured approach helps in designing scalable systems.



BENEFITS

- ✓ Helps build robust systems
- ✓ Handles real-world complexity
- ✓ Improves communication

KEY FOCUS

- Scalability
- Reliability
- Performance
- Maintainability



INTERVIEW TIP: Practice designing end-to-end systems for common use cases.



REMEMBER:

Understand the problem, design smart, and build for scale. !
Great systems solve today and prepare for tomorrow!





SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 13 of 15 •



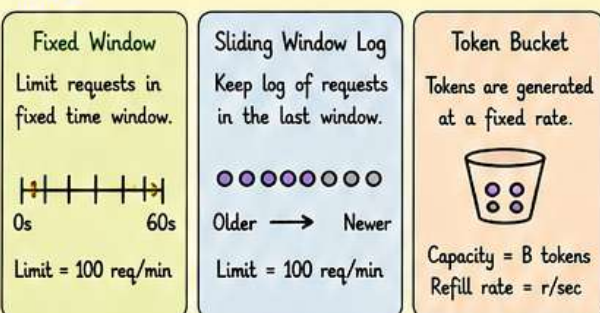
Measure what matters, learn continuously,
and make data-driven decisions!



47 RATE LIMITING

Rate limiting restricts the number of requests a user or client can make in a given time.

TYPES



BENEFITS

- ✓ Protects from abuse
- ✓ Ensures fair usage
- ✓ Improves system stability

USE CASES

- API rate limiting
- Login attempts
- Prevent DDOS



INTERVIEW TIP: Discuss algorithms, data structures (Redis), and trade-offs between approaches.



48 SHUFFLE / RANDOMIZATION

Randomization is used in load balancing, A/B testing, recommendations, sampling, etc.

COMMON TECHNIQUES

- Fisher-Yates Shuffle (Uniform shuffle of array)
- Consistent Hashing (For stable distribution)
- Reservoir Sampling (Sample k items from stream)

FISHER-YATES SHUFFLE EXAMPLE

Input : [1, 2, 3, 4, 5]
 $i = 4 \rightarrow \text{swap}(4, \text{random}[0..4])$
 [1, 2, 3, 5, 4]
 $i = 3 \rightarrow \text{swap}(3, \text{random}[0..3])$
 [1, 2, 4, 5, 3]
 $i = 2 \rightarrow \text{swap}(2, \text{random}[0..2])$
 [1, 4, 2, 5, 3]
 $i = 1 \rightarrow \text{swap}(1, \text{random}[0..1])$
 [4, 1, 2, 5, 3]

USE CASES

- ✓ Load balancing
- ✓ A/B testing
- ✓ Recommendations
- ✓ Distributed caching

BEST PRACTICES

- Ensure uniform distribution
- Avoid predictable patterns
- Use good random source

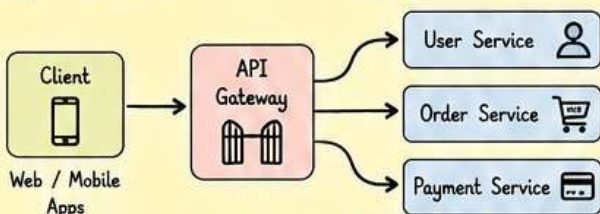


INTERVIEW TIP: Talk about randomness quality, consistency, and when each technique is used.



49 API GATEWAY

API Gateway acts as a single entry point for all client requests and routes them to the appropriate service.



RESPONSIBILITIES

- ✓ Request routing
- ✓ Authentication & Authorization
- ✓ Rate limiting
- ✓ Caching
- ✓ Monitoring & Logging
- ✓ Response aggregation

BENEFITS

- Simplifies clients
- Centralized control
- Security enforcement
- Protocol translation
- Observability



INTERVIEW TIP: Compare with Service Mesh. When to use API Gateway vs BFF?



50 SYSTEM DESIGN TRADE-OFFS

Every design decision has trade-offs. Choose based on requirements and constraints.

TRADE-OFF	OPTION A	OPTION B	WHAT TO CONSIDER
Consistency	Strong Consistency	Eventual Consistency	Data correctness vs availability
Availability	High Availability	Lower Availability	Uptime vs strict consistency
Latency	Low Latency	High Latency	User experience vs complexity
Scalability	Vertical Scaling	Horizontal Scaling	Cost, complexity, and limits
Cost	Low Cost	High Cost	Budget vs performance



INTERVIEW TIP: Always justify your choice with real-world implications.



REMEMBER: There is no perfect design, only the right design for the right problem at the right time! ❤️



SYSTEM DESIGN

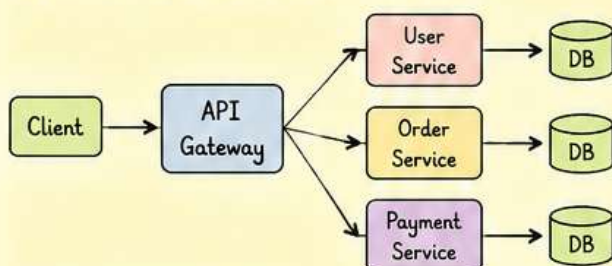
INTERVIEW QUESTIONS

• Page 14 of 15 •

★ Scalable systems are not accidental, they are designed with patterns and principles! ❤️

51 MICROSERVICES

Breaks a large application into small, independent services that communicate over a network.



BENEFITS

- ✓ Independent deployment
- ✓ Fault isolation
- ✓ Technology freedom
- ✓ Easy to scale

CHALLENGES

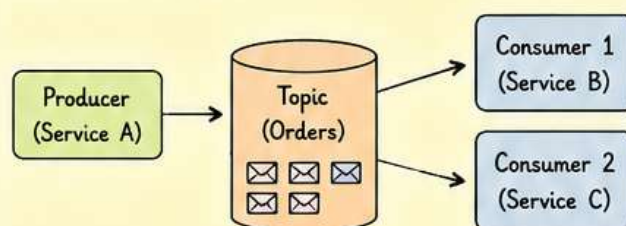
- ✗ Complexity
- ✗ Distributed data
- ✗ Network latency
- ✗ Harder debugging



INTERVIEW TIP: When to choose microservices over monolith? What are the trade-offs? 🛡️

52 MESSAGE BROKER (KAFKA)

A distributed system that decouples producers and consumers using topics.



BENEFITS

- ✓ Decoupling
- ✓ High throughput
- ✓ Durability
- ✓ Scalability

USE CASES

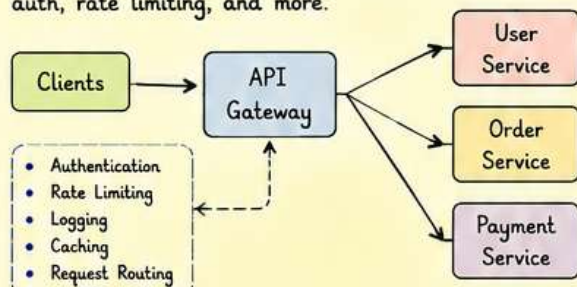
- Event streaming
- Activity tracking
- Log aggregation
- Real-time analytics



INTERVIEW TIP: Difference between Kafka and RabbitMQ? 🛡️

53 API GATEWAY

Single entry point for all client requests. Handles routing, auth, rate limiting, and more.



BENEFITS

- ✓ Simplifies clients
- ✓ Centralized cross-cutting concerns
- ✓ Improves security
- ✓ Better monitoring

CHALLENGES

- ✗ Single point of failure
- ✗ Adds latency
- ✗ Extra infrastructure

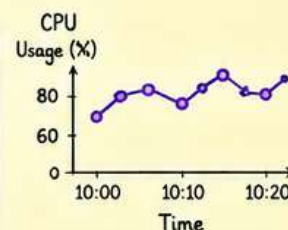


INTERVIEW TIP: What are common features of an API Gateway? 🛡️

54 TIME SERIES DATABASE

Optimized for storing and querying time-stamped data at scale.

timestamp	metric	value
2024-05-01 10:00:00	CPU	65
2024-05-01 10:00:10	CPU	70
2024-05-01 10:00:20	CPU	68
...	...	...



BENEFITS

- ✓ High write performance
- ✓ Efficient compression
- ✓ Fast queries on time range
- ✓ Built-in retention policies

EXAMPLES

- InfluxDB
- TimescaleDB
- Prometheus
- OpenTSDB



INTERVIEW TIP: Where would you use a time series DB? 🛡️



REMEMBER: Every system has trade-offs.

Understand them, justify your choices, and keep improving! ❤️



SYSTEM DESIGN

INTERVIEW QUESTIONS

• Page 15 of 15 •

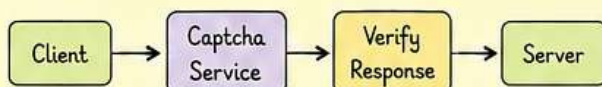


Keep learning, keep building, and keep designing amazing systems!



55 CAPTCHA SERVICE

Prevents bots from abusing systems and protects critical endpoints.



BENEFITS

- ✓ Stops automated bots
- ✓ Protects signups / logins
- ✓ Reduces spam
- ✓ Improves security

CHALLENGES

- ✗ User friction
- ✗ Accessibility
- ✗ Solving at scale
- ✗ Bypass attempts

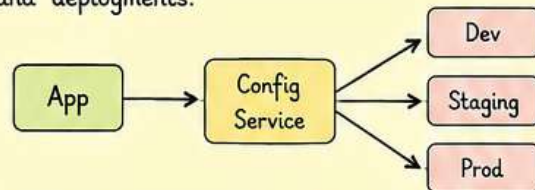


INTERVIEW TIP: When and where would you use CAPTCHA and what are the alternatives?



56 CONFIGURATION MANAGEMENT

Manage configuration for environments, services and deployments.



BENEFITS

- ✓ Centralized configs
- ✓ Easy environment mgmt
- ✓ Consistency
- ✓ Safe updates

BEST PRACTICES

- Version configs
- Environment isolation
- Secrets management
- Audit & rollback

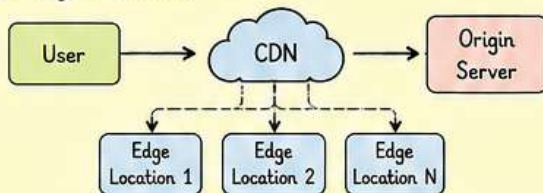


INTERVIEW TIP: How do you manage secrets and environment-specific configs?



57 CDN (CONTENT DELIVERY NETWORK)

Delivers content globally with low latency by caching at edge locations.



BENEFITS

- ✓ Low latency
- ✓ Reduces origin load
- ✓ High availability
- ✓ Better user experience

USE CASES

- Static assets (JS, CSS, Images)
- Video streaming
- Software downloads
- Global websites



INTERVIEW TIP: How does CDN caching, invalidation and TTL work?



58 SERVICE DISCOVERY

Allows services to find and communicate with each other in a dynamic environment.



Registry Examples

- Eureka
- Consul
- etcd
- Zookeeper

BENEFITS

- ✓ Dynamic scaling support
- ✓ No hard-coded endpoints
- ✓ Resilience
- ✓ Easier deployments

CHALLENGES

- Registry as SPOF
- Consistency
- Network partitions
- Security



INTERVIEW TIP: Compare Eureka vs Consul vs Zookeeper.



FINAL CHEAT SHEET

- ★ Design for scale and failure.
- ★ Understand trade-offs, not just technologies.
- ★ Always clarify requirements first.
- ★ Use the right tool for the right problem.
- ★ Keep it simple, then optimize.

- ★ Think in components, not code.
- ★ Communicate your design clearly.
- ★ Consider performance, cost and reliability.
- ★ Iterate and improve continuously.

Happy Designing! ❤️



REMEMBER: Great design comes from practice and curiosity.
Keep building, keep questioning, keep growing! ❤️